InsightFlow-Agent2 技术说明书（交付评审用）

> **读者**：技术负责人 / 评审人。 **目的**：用一份文档说明"业务需求是什么、功能怎么实现、架构为什么这样、出错怎么补救、工程化与安全做到了什么程度"， 以便判断是否值得投入、以及投入前必须先解决什么。 **原则**：文中每个数字都能在仓库或日志中复核；没做到的会明确写"未做"。本文档不包含任何夸大性结论。

---

0. 一页摘要

项	事实
形态	单机部署的 Web 服务 + 控制台，Python 3.11
业务	把"看数据、写结论"的重复劳动变成：上传表 → 提固定问题 → 带图、带口径、可追溯的报告 → 自动推送到群
编排	LangGraph，23 个节点；固定循环（修复/精修/绘图各 3 次）+ 运行期动态决策（自带硬上限）
代码量	生产代码 28,846 行 / 144 文件；测试 27,941 行 / 92 文件（1,102 个 `def test_`，83 个测试文件）
存储	双模式：`local`（SQLite + 进程内向量）与 `postgres`（PostgreSQL + pgvector + Redis）
安全	Docker 沙箱（禁网/只读根/非 root/限额）、SQL 只读五重约束、Bearer 鉴权与 owner 隔离、上传类型与大小限制
成熟度	工程链路完整、有回归与真实模型验证；**但基于脚本化模型的 1158 用例只证明工程正确性，不证明模型能力**

---

1. 业务需求

1.1 场景与角色

使用者	1–5 人小团队里兼做数据的人（运营、客服主管、店长），**不是**专业分析师
频率	每周/每天固定几次，问的是同类问题
现状	导出 CSV → Excel 透视 → 挑数 → 手写结论，单次约 90 分钟
三个真实痛点	① 只来得及看总量，看不出"哪个地区/品类在悄悄掉"；② 结论没有出处，被追问要重算；③ 遇到脏数据（月份不完整、运费被当成商品）会得出错误结论且不自知

1.2 需求清单与实现对应

#	需求	实现	验收证据
R1	接受 CSV/Excel/SQLite，不必改造成数仓	`ingestion/`：加载为内存表 + 可序列化快照，SQL 走 SQLite 临时库	`test_tools.py`、`test_sql_hardening.py`
R2	把问题拆成可执行的分析任务	`agents/planner.py` + `planning/validator.py`	`test_plan_validator.py`
R3	任务要真的被执行（Python 或 SQL）	`router` 按来源类型选工具；`tools/python_tool.py`、`sql_tool.py`	`test_api_workflow.py`
R4	结果不对时能自己修	固定循环：修复 3 次、精修 3 次、绘图修复 3 次	`test_dynamic_loop.py`
R5	结论要能追到来源	任务级检索证据（文档/页码/chunk/分数/版本）+ 报告"参考来源"附录	`test_task_aware_rag.py`、`test_evidence_isolation.py`
R6	业务口径必须生效（否则数字是错的）	口径写成知识文档，随分析检索注入；报告要求写明口径	见 §7.3 对照实验
R7	人工可介入（执行前/发布前）	LangGraph `interrupt` 两个关口 + 计划指纹校验	`test_human_in_the_loop.py`、`test_report_approval.py`
R8	跑完自动通知，不用守着	`integrations/feishu.py` + `services/run_notifier.py`（守护线程发送）	`test_feishu_notifier.py`
R9	中断后不能白跑	Checkpoint + 操作幂等 + 账本；跨进程恢复	`test_cross_process_recovery.py`、`test_dynamic_recovery.py`
R10	花销有上限	预算按**真实上游请求**计数，前置于请求，随 Checkpoint 持久化	`test_budget_real_entry_points.py`、`test_llm_budget_accounting.py`
R11	多人可用且互相看不见	Bearer 令牌 → 归属者；会话/记忆按 owner 过滤	`test_auth_owner.py`
R12	用户的表能传进来	`POST /api/uploads`（类型/大小/路径穿越三重限制）	`test_upload_api.py`

1.3 明确不做（当前版本）

- 不做通用 BI / 实时大屏 / ETL 平台 / 多租户 SaaS。
- 不做自然语言查任意数据库（只读 + 白名单表 + 词法校验）。
- 不做模型微调；只做提示与上下文工程。

---

2. 功能实现（端到端）

2.1 主链路

```
loader → context → knowledge(全局检索) → planner → 计划校验
       → [approval_gate] → router → execution → tool_execution
       → critic → decide(动态决策) → advance → visualization_planner
       → plot_generation → plot_execution → report → artifact
```

23 个节点，`workflow.py` 中 `add_node` 计数可复核。

2.2 各能力的实现要点

能力	实现要点
规划	输出结构化任务（`task_id` / `description` / `target_tables` / `expected_output` /可选 `depends_on`）
计划校验	程序检查：必填字段、ID 唯一、目标表必须是**本次 Run 的授权资源**、来源类型可落到单一工具、依赖不成环；非法则携带原因重写，最多 3 次
执行	生成代码 → 沙箱容器执行；大结果外置为带 sha256 的产物引用，state 只留摘要
审查	Critic 判断结果是否回答了任务；不通过进入精修（有上限）
动态决策	构造观察（结果形状/错误分类/预算余量）→ 模型在 7 个动作中选择 → 确定性策略校验后执行
可视化	规划图表 → 生成绘图代码 → 沙箱执行 → 失败修复（3 次）→ 未生成的图写入报告"未能生成的图表"
报告	模型生成正文 + 系统追加"参考来源""图表"附录；Markdown/HTML/PDF 三份产物
多轮会话	会话 → 轮次 → Run；历史摘要注入；长期记忆候选-确认两阶段
审批	`interrupt` 中断；恢复用 `Command(resume=...)`；计划指纹防止"审批后换计划"

---

3. 架构设计

3.1 分层与依赖方向

```
api*.py                传输层：校验输入、映射状态码、鉴权中间件
services/              编排层：RunCoordinator、会话、记忆、知识、通知
workflow/nodes         图：拓扑、路由、Checkpoint、中断
harness/               治理层：每次模型/工具调用的唯一入口（预算、账本、幂等、上下文）
agents/ planning/ agent_loop/ rag/ storage/    能力层
tools/ sandbox/ mcp_bridge/                    执行层
```

硬约束：Agent 节点不得直接写 SQL、不得直接访问 Redis、不得自行构造模型客户端；一切模型与工具调用经 `harness/`。这样预算、账本、幂等在系统中只有一份实现。

3.2 状态设计

- `state.InsightFlowState` 是 Checkpoint 契约：所有值必须 JSON 可序列化。
- **不进 state**：数据行（按 `table_snapshot` 读回）、模型客户端、DataFrame。
- **必须进 state**：任务进度、失败任务、预算快照、决策历史、已完成操作登记、产物路径。
- 新增字段必须同时在 TypedDict 声明，否则 LangGraph 会静默丢弃（本项目真实踩过，见 §6.7 事故 9）。

3.3 存储双模式

	`local`	`postgres`
Run 登记	SQLite	PostgreSQL `runs`
Checkpoint	SqliteSaver	PostgresSaver
知识索引	进程内	pgvector + hnsw
会话/记忆	SQLite	PostgreSQL
缓存	关闭	Redis（可选）

实测 `postgres` 模式的表：`runs`、`checkpoints`、`checkpoint_blobs`、`checkpoint_writes`、`conversation_*`、`long_term_memories`、`knowledge_*`、`run_jobs`、`session_run_links`、`schema_migrations`。

3.4 部署形态

形态	适用	说明
本机	开发/演示	`scripts/serve_api.py`，SQLite，零外部依赖
混合（当前使用）	Windows	API 跑宿主 + PostgreSQL/Redis 进容器；因为分析要在宿主 Docker 沙箱执行
全容器	Linux	`deploy/compose.app.yaml`（api + worker + pg + redis）；需挂 docker.sock 才能执行 Python 分析

4. 技术选型与理由

选择	备选	为什么选它	代价/风险
LangGraph	自研状态机、Temporal、Celery 编排	需要"图 + 中断 + 持久化 checkpoint"三件事同时具备；`interrupt` / `resume` 是 HITL 的关键能力	框架较新，API 有变动风险；Checkpoint 契约受其实现约束
LangChain Core	直接用 OpenAI SDK	统一消息/工具/结构化输出抽象，换 provider 成本低	抽象层偶有隐式行为（如结构化输出解析）
FastAPI	Flask、Django	类型化请求/响应 + 自动 OpenAPI，控制台与评测脚本都直接受益	同步端点下长任务必须自己管线程/队列
Pydantic	dataclass + 手写校验	请求校验与错误信息开箱可用	v2 迁移成本（本项目已用 v2）
PostgreSQL + pgvector	专用向量库（Milvus/Qdrant）、纯内存索引	会话、记忆、知识、Run 状态共用一个库，少一套运维；向量规模在几千 chunk 级别，pgvector 足够	大规模向量检索（百万级）性能不如专用库
Redis	只用内存缓存	检索缓存跨进程共享，重启不丢热度	多一个依赖；因此设计为**可关闭**且不影响正确性
SQLAlchemy 2 + psycopg3	裸 psycopg	两种后端同一套模型；迁移可控	ORM 学习成本
Docker 沙箱	子进程 + 受限 builtins、Firecracker	容器级隔离（禁网、只读根、非 root、限额）比语言级限制可靠	依赖宿主 Docker；Windows 上容器内 API 无法调用
MCP（stdio）	自研插件协议	把 SQL 执行放到独立进程，按 Run 授权；协议标准化便于替换	多一层进程与协议开销；默认仍用 direct
pytest	unittest	fixture/参数化/标记适合"真实容器 + 跨进程"混合测试	无（但缺 CI 编排）
feishu webhook	自建应用 + 回调	群机器人零门槛、不需要公网回调	只能单向推送，卡片按钮需自建应用
sentence-transformers（本地）	云 embedding API	离线、无调用成本、数据不出机器	首次加载慢、模型能力弱于商用
PDF 用浏览器打印	wkhtmltopdf、WeasyPrint	与 HTML 报告同源，样式一致	依赖本机 Chrome/Edge；容器内需另装，否则降级

5. 工程化

5.1 目录与职责

`agents/ agent_loop/ planning/ harness/ rag/ storage/ services/ tools/ sandbox/ mcp_bridge/ ingestion/ context/ core/ utils/ web/ deploy/ docs/ data/ knowledge/ tests/ scripts/`

5.2 配置管理

- 单一来源：`.env`（gitignore），模板 `.env.example` 共 20 个键。
- 分类：模型凭据 3 个、存储 4 个、缓存 3 个、归属 1 个、鉴权 1 个、飞书 3 个、compose 5 个。
- **配置错误要早失败**：`INSIGHTFLOW_STORAGE` 非法、`queue` 模式配 local 存储、`INSIGHTFLOW_API_KEYS` 格式错误，都会在启动/请求时明确报错，而不是静默退化。
- 提供 `scripts/set_env_value.py`：只改单个键，避免整体重写把别的配置抹掉（真实发生过）。

5.3 依赖与可复现

- `requirements.txt` 声明全部导入期依赖（含 SQLAlchemy/psycopg/redis/pgvector/checkpoint-postgres）。
- 曾在全新 venv 中实测：修复前 `import api` 直接 `ModuleNotFoundError`；修复后 `pip install -r requirements.txt` → 启动 → `/health` 通。
- 未做哈希锁定（无 lock 文件）：可复现的是"可运行"，不是"依赖树逐字节相同"。

5.4 测试策略

层	做法
单元	纯函数与策略（预算、策略校验、作用域分类）
契约	脚本化模型驱动真实图执行，断言外部可观察结果
真实依赖	PostgreSQL 容器、Redis 容器、Docker 沙箱、MCP 子进程
恢复	真子进程 `os._exit` 强杀，再重启校验
判别性	每个确认缺陷先写失败用例，修复后转绿（`test_quality_regressions.py`）
网络	全局 network guard，防止测试误打真实模型

规模：1,102 个 `def test_`（83 个文件，参数化后运行时更多）。最近一次全量运行：**1158 通过 / 2 跳过，约 26 分 40 秒**（跳过项为"本机不允许创建符号链接"与"未设置 INSIGHTFLOW_EVAL_OUT"）。

5.5 观测性

- 结构化 JSONL 日志（每次 Run 一个文件）：节点、事件、状态、耗时、上下文用量。
- 操作账本：每个逻辑操作 `started/completed` 两条记录。
- 预算快照：`limits` + `usage`（轮次/工具/模型请求/执行秒数）。
- 对外状态投影：`/runs/{id}/state` 只暴露可观测字段（含失败任务、失败图表、决策历史、预算）。
- 健康检查：`/health`、`/health/ready`、`/storage/health`（含各后端与表就绪状态）。

5.6 提交与发布

- 分支：`main`（可用）+ `review/p2d-p5-clean`（开发分支，与 main 同步）。
- 提交信息写"为什么"，不是"改了什么"。本次交付共 62 次提交（含早期历史）。
- **未做**：CI（无 GitHub Actions）、无自动化发布、无覆盖率统计。

---

6. 出错与补救

6.1 错误分类（决定"修"还是"停"）

类别	例子	处理
可修复	代码异常、SQL 语法错、结果未回答问题	进入修复/精修循环，有上限
终止	权限拒绝、授权失败、环境故障（MCP 不可用、数据源缺失）、预算耗尽	**禁止重试**，直接跳过该任务或收尾
参数/配置	非法 sql_backend、非法执行模式、上传类型不支持	400/500 明确返回，不进工作流

6.2 有界重试

循环	上限	位置
初始计划重写	3	`nodes.MAX_PLAN_ATTEMPTS`
执行修复	3	`conditions.MAX_RETRIES`
质量精修	3	`conditions.MAX_REFINES`
绘图修复	3	`conditions.MAX_PLOT_RETRIES`
动态决策步数 / 重规划 / 补检索 / 重复动作	8 / 2 / 2 / 2	`agent_loop/policy.py`
预算总量	轮次 24 / 工具 40 / 模型 60 / 1800 秒	`utils/agent_loop.AgentBudget`

6.3 崩溃恢复

- **Checkpoint**：每个节点执行后提交；恢复从最后提交点继续。
- **幂等**：写产物、工具调用带 `operation_id`，恢复时命中登记即跳过。
- **账本**：模型调用"已发出未确认"会被记录，由人工决定是否重发。
- **验证**：3 个窗口（决策后、工具后、计划修订后）真子进程强杀 + PostgreSQL 重连恢复，均有自动化用例。

6.4 恢复语义（明确不说满）

至少一次，不是恰好一次：模型响应已返回但节点未提交时，恢复会重发该请求。代码里有一组用例专门确认"我们没把这条说成 exactly-once"。这对计费敏感场景是必须知道的事实。

6.5 降级清单

场景	行为
PDF 导出失败（无浏览器）	记录 `degraded`，Run 仍完成，交付 Markdown + HTML
飞书推送失败	守护线程内吞掉异常并记日志，Run 不受影响
Redis 不可用	检索直接走持久索引（自动降级，只是变慢）
图表失败	写入 `failed_charts`，报告列出"未能生成的图表"
队列 worker 崩溃	任务停在 `claimed`，需人工 `requeue`（**不自动重试**）

6.6 失败可见性

- 任务级：`failed_tasks`（含阶段与原因）→ 报告与状态都可见。
- 图表级：`failed_charts` → 报告小节 + `/state` 投影。
- Run 级：`status_reason`、`error`。
- 报告级：附录（参考来源、图表、未能生成的图表）。

6.7 真实事故与修复（摘要，完整版见仓库外 `real-model-errors.md`）

#	现象	根因	修法
1	图表 0/3，`ImportError: __import__ not found`	沙箱禁 `__import__`，模型固定写 `import pyplot`	错误改为可执行提示；后升级为**导入白名单**（34 个模块）
2	`NameError: isinstance/print`	内置函数白名单过窄	补 70 个安全内置；`print` 重定向到 stderr
3	`KeyError: 'hv_sales_contribution'`	模型猜结果键名	把 `analysis_result` **真实结构**写进绘图/修复提示词
4	图表失败但报告不提	`failed_charts` 写进节点返回值但 state 未声明，被 LangGraph 丢弃	在 TypedDict 声明；并加用例断言声明存在
5	报告发布后同一次运行状态显示 `partial`	空计划/失败任务判定顺序	统一由 artifact 节点从 `failed_tasks` 推导
6	部署时 `/storage/health` 说 postgres，实际写 SQLite	`serve_api.configure` 写死 SQLite	改为按配置构造 store 与 checkpointer
7	模型请求上限 3 却发出 4 次	节点只返回部分 state，累计用量回退	预算增量随节点返回值一起写回；用例钉住"上限 N = N 次"
8	飞书卡片显示字面量反引号	`lark_md` 不支持行内代码	去掉反引号；加可选控制台链接
9	通知拖慢 Run，导致审批被判"并发执行"	通知同步发送	改为守护线程；测试隔离 FEISHU_*
10	12 月不完整却算出 −74% 环比	缺业务口径	口径写成知识文档随分析注入（见 §7.3）

---

7. 安全

7.1 沙箱隔离

维度	做法
网络	`--network none`
文件系统	只读根 + 只挂载本次 Run 的输出目录（可写）
权限	`--user` 非 root、`--cap-drop`、`--security-opt`
资源	内存/内存+swap/CPU/pids 限制
语言级	内置函数白名单（70 项，无 `open/eval/exec/compile/setattr`）；导入白名单（34 个纯计算与绘图模块，无 `os/subprocess/socket/pathlib/shutil/pickle`）

7.2 SQL 边界

词法校验（拒绝多语句/写操作）→ 只读连接 → SQLite Authorizer → 查询超时 → 结果行数/大小上限。表名必须是本次 Run 的授权资源。

7.3 鉴权与数据隔离

- `INSIGHTFLOW_API_KEYS=token:owner,...`；**不配置即单用户演示模式**（默认关闭鉴权，便于本地使用）。
- 归属者只从 token 推导，请求体无 owner 字段。
- 越权读取他人会话返回 **404**（而非 403），避免泄露"该 id 存在"。
- 报告与图表路由公开：浏览器点链接不带 Authorization，且 run_id 随机不可枚举。
- 已知边界：这是"令牌 → 归属者"映射，**不是完整身份系统**；没有注册、找回报价、配额与账单。

7.4 上传

类型白名单（csv/xlsx/sqlite/db/sqlite3/md/txt/pdf）→ 单文件 20 MB → 只取 basename（`../../api.py` 被削成 `api.py`）→ 目录由服务端决定 → 超限时删除半成品。

7.5 凭据

`.env` 不进版本库（历史中也从未提交）；`.env.example` 只有空占位；`scripts/check_model.py` 用于区分 401/402/404；`scripts/set_env_value.py` 只改单个键。

7.6 已知安全风险

1. 队列模式需要容器访问宿主 Docker（`docker.sock`），等价于把 Docker 权限交给该容器——**仅适合受信环境。**
2. 无速率限制：持有 token 的调用方可无上限提交 Run（只有单次预算，没有按用户/按天配额）。
3. 上传文件不扫描内容（只校验扩展名与大小）。
4. 沙箱依赖宿主内核隔离，不是 VM 级隔离。

---

8. 质量现状（可复核）

项	数值	来源
全量用例	1158 通过 / 2 跳过（约 26 分 40 秒）	最近一次全量运行日志
代码规模	生产 28,846 行 / 测试 27,941 行	`collect_facts.py` 统计
节点数	23	`workflow.py`
真实模型验证	小数据 3/3 图、零售 17,049 行 3/3 图	运行日志 + 报告
结论正确性	GMV 环比 +1.98%（标准答案 +2.0%）；退货冠军 `22423` 与标准答案一致	`case_ground_truth.json` 对照

8.1 真实模型能做什么、不能证明什么

能证明：模型能在这套约束下把 1.7 万行数据跑成一份结构完整、口径正确、带图与来源的报告。 **不能证明**：模型在**没给口径**时也不会出错——它错过（−74%），这正是要"口径进知识库"的原因。

---

9. 交负责人评估时的建议路径

1. **先看业务成立性**：`docs/case-study.md`（场景 + 标准答案 + 对照实验），5 分钟。
2. **再看架构边界**：`harness/`（唯一入口）、`agent_loop/policy.py`（决策边界）、`planning/validator.py`（计划校验）。
3. **再看错误处理**：`services/run_coordinator.py` 的 `_finalize` / `_fail`、`utils/agent_loop.py` 的预算、`services/run_queue.py`。
4. **最后验证**（三条命令）： `bash python scripts/check_model.py python scripts/real_model_smoke.py python scripts/real_model_regression.py --include-retail`

---

10. 已知限制与风险（按影响排序）

#	限制	影响	建议
1	评估基于脚本化模型，真实模型只验过 2 个场景	不能据此承诺准确率	接入更多真实数据集做基准
2	无速率限制与按用户配额	单用户可耗尽额度	加按 token 的日配额
3	队列无自动重试、无 worker 心跳回收	崩溃后任务需人工 `requeue`	加超时回收 + 重试策略
4	单机、无线程/进程级隔离扩展	无法水平扩展	引入外部队列（如 Redis Streams）与多 worker
5	沙箱依赖宿主 Docker	部署耦合	评估 gVisor / 远程沙箱服务
6	无 CI、无覆盖率门槛	回归靠手工	加 GitHub Actions + 覆盖率阈值
7	依赖未锁定哈希	跨机器可能装到不同小版本	引入 lock 文件
8	多租户缺身份系统	只适合内部小范围	接企业 SSO（Casdoor/飞书）

11. 后续路线（若继续投入）

优先级	事项	说明
P0	速率限制与按用户配额	没有它，公开部署等于敞开额度
P0	队列自动回收 + 有限重试	当前需人工干预
P1	CI + 覆盖率	让回归可信任
P1	真实数据基准集（3–5 个场景）	让"能分析"变成可度量
P2	多租户身份（SSO）	从"小团队内部"走向"可交付客户"
P2	沙箱替换为 gVisor/远程执行	解耦宿主 Docker

附：本文档未包含的内容

- 未包含任何真实客户数据、真实密钥或内部路径。
- 未包含性能压测结论（未做）。
- 未包含与竞品的对比评测（未做）。